SHENai PRIVATE SOFTWARE SOLUTION

ARTIFICIAL INTELLIGENCE

THREE MONTHS

An INTERNSHIP REPORT

Submitted by

MADHUSHREE M

In partial fulfillment for the award of the degree

of

BACHELOR OF TECHNOLOGY

in

ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

SSM INSTITUTE OF ENGINEERING AND TECHNOLOGY

DINDIGUL 624 002

OCTOBER 2026

BONAFIDE CERTIFICATE

This is to certify that **MADHUSHREE (922123243031),** is a bonafide student of AI&DS Department, SSM Institute of Engineering and Technology, Dindigul – 624 002, affiliated to Anna University.

The student is studying in VII Semester during the academic year 2026–2027 and has undertaken the Industry Internship at **SHENai private software solution**sfrom 05-07-2026 to 05-09-2026 as part of the academic requirements of the programme.

SIGNATURE
Dr.M.PREMKUMAR
HEAD OF THE DEPARTMENT
Department of Artificial Intelligence and Data Science,
SSM Institute of Engineering and Technology,
Dindigul - 624002.

SIGNATURE
Ms. I . JOICY
ASSISTANT PROFESSOR
Department of Artificial Intelligence and Data Science,
SSM Institute of Engineering and Technology,
Dindigul - 624002.

BONAFIDE CERTIFICATE

This is to certify that **Madhushree M** is a bonafide student of **SSM Institute of Engineering and Technology, Dindigul**, pursuing **B.Tech in Artificial Intelligence and Data Science**.

The student completed an internship in the **Artificial Intelligence (AI) domain** at **SHENai Private Software Solutions, Erode**, from **5 July 2026 to 5 September 2026**. During the internship, the student worked on **Automated Python Docstring Generator** under the guidance of the industry mentor, as applicable.

The internship details were collected and verified at the departmental review meeting held on [22-10-2026]

Company Address: Erode, Tamil Nadu, India.

Industry Mentor: S. SHYSAM, AI ENGINEER.

ACKNOWLEDGEMENT

First and foremost, we would like to express our deep sense of gratitude to our most honorable Chairman and Management Trustees **Mr. K. Shanmugavel** for providing us with necessary facilities during the course of study. We feel immensely pleased to express our sincere thanks to our Principal

Dr. S. Karthigai Lakshmi for encouragement and support extended by them. We extend our solemn gratitude to **Dr. M. Premkumar**, Head of the Department, Artificial Intelligence and Data Science, for timely support to all our activities. I would like to thank my Faculty Internship Coordinator **Ms. SANGEETHA MARY, AP/AI&DS** for the continuous guidance, valuable suggestions, and encouragement during the internship and in the preparation of this report.

I am grateful to **SHENai PRIVATE SOFTWARE SOLUTION** for providing me with the opportunity to gain practical exposure to an industry environment. I sincerely thank my Industry Mentor, **S.SHYAM SATHYAMOORTHY**, and the entire team for their guidance, technical support, and valuable insights throughout the internship.

We express our sincere thanks to our guide **Ms. I. Joicy, AP/AI&DS** for her guidance throughout our Internship. Also we express our thanks to the faculty members of our department, non-teaching staff members and my dear friends for their moral support, help and encouragement towards the successful completion of the internship.

INDEX

S.NO	CONTENTS	PAGE NO
1.	Bonafide Certificate	2
2.	Internship Confirmation Letter	3
3.	Declaration	5
4.	Introduction	8
5	Company Profile	10
6	Internship Activities	15
7	Major Project	19
8	Skills and Learning Outcomes	25
9	Challenges and Solutions	29
10	Professional Experience	32
11	Conclusion	35
12	Future Scope	38
13	Appendices	40
14	Internship Completion Certificate	45

Internship Confirmation Letter

SHENai
Private Software Solutions

Erode, Tamil Nadu, India

INTERNSHIP OFFER LETTER

Date of Issue: 01 July 2026

To,
Madhushree M
B.Tech – Artificial Intelligence and Data Science
SSM Institute of Engineering and Technology, Dindigul

We are pleased to offer you an internship opportunity at SHENai Private Software Solutions. This internship will provide you with practical exposure in the Artificial Intelligence domain and includes hands-on experience in Python programming, AI integration, and software application development.

INTERNSHIP DETAILS

Internship Domain	:	Artificial Intelligence (AI)
Start Date	:	5 July 2026
End Date	:	5 September 2026
Internship Duration	:	Approximately two months

During the internship, you will have the opportunity to apply theoretical concepts to practical project development, improve technical and problem-solving skills, and gain exposure to professional software development practices.

We look forward to your positive response and a successful learning journey with us.

Sincerely,
Senthamarai M
Founder & CEO
SHENai Private Software Solutions

EXECUTIVE SUMMARY

This internship report presents an overview of the internship undertaken at **SHENai Private Software Solution,** in the domain of **Artificial Intelligence (AI)**. The internship provides an opportunity to understand the practical applications of artificial intelligence and to gain exposure to industry-oriented software development practices.

The primary purpose of the internship is to bridge the gap between academic knowledge and practical industry experience. It helps in understanding the fundamental concepts of artificial intelligence, exploring AI-based tools and technologies, and learning how intelligent solutions can be developed to address real-world problems. The internship also provides an opportunity to understand the importance of problem-solving, logical thinking, and systematic approaches in software development.

During the internship, the focus is on gaining knowledge of AI concepts, understanding the development workflow, exploring relevant technologies, and improving technical skills through practical learning activities. It also helps develop professional skills such as communication, teamwork, time management, and adaptability to a professional environment.

Overall, the internship serves as a valuable learning experience that strengthens the understanding of artificial intelligence and its applications. The knowledge and skills gained through this experience can support future academic projects and career opportunities in Artificial Intelligence, Machine Learning, and related software development fields.

INTRODUCTION

BACKGROUND

Artificial Intelligence (AI) is a technology that helps computers perform tasks such as learning, problem-solving, and decision-making. It is widely used in many fields to make work easier and faster.

The internship at **SHENai Private Software Solution, in** the Artificial Intelligence domain, helps students understand AI concepts and their practical applications. It provides an opportunity to improve technical knowledge, learn new technologies, and understand how AI is used to solve real-world problems.

This internship also helps improve problem-solving, communication, and teamwork skills, which are useful for future career development.

INTERNSHIP OVERVIEW

The primary objective of this internship was to gain practical knowledge of AI concepts and understand their applications in real-world scenarios.

The internship provided an opportunity to enhance technical knowledge, explore AI-related technologies, and understand software development practices. It also contributed to the development of analytical thinking, problem-solving abilities, and professional skills.

Overall, the internship served as a valuable learning experience by connecting academic knowledge with practical applications and providing a foundation for future career opportunities in Artificial Intelligence and related fields.

NEED AND PURPOSE

It helps students explore AI-related technologies, learn about industry practices, and understand how technological solutions can be developed to address real-world challenges.

The purpose of the internship is to enhance technical knowledge, improve analytical thinking, and develop problem-solving abilities. It also helps students understand professional responsibilities, communication, teamwork, and time management. Through this learning experience, students can gain a better understanding of industry expectations and prepare themselves for future employment opportunities.

Overall, the internship serves as a bridge between academic education and professional practice. It supports skill development, encourages continuous learning, and provides a foundation for further studies and career growth in Artificial Intelligence and related technology fields.

OBJECTIVE OF THE INTERNSHIP

1. To understand the fundamentals of Artificial Intelligence
2. To gain practical technical knowledge
3. To understand real-world applications
4. To improve analytical and problem-solving skills
5. To gain exposure to software development practices
6. To develop professional skills
7. To connect academic knowledge with industry practices
8. To prepare for future career opportunities

SCOPE OF THE INTERNSHIP

The scope of the internship focuses on understanding the concepts, technologies, and applications associated with Artificial Intelligence. It provides

an opportunity to explore the role of AI in software development and learn how intelligent systems can support problem-solving and improve work efficiency.

The internship includes gaining theoretical and practical knowledge of AI concepts, exploring relevant technologies, and understanding the basic processes involved in developing technology-based solutions. Depending on the activities assigned during the internship, the learning process may include understanding requirements, analysing problems, exploring suitable tools, implementing solutions, and evaluating the results.

Another important aspect of the internship is the development of technical and professional competencies. Students can improve their analytical thinking, logical reasoning, communication, teamwork, and time management skills. These competencies are important for working effectively in technology-oriented environments.

The internship also helps students understand the importance of continuous learning in the rapidly developing field of Artificial Intelligence. As AI technologies continue to evolve, knowledge of current tools, applications, and development practices becomes increasingly valuable.

COMPANY PROFILE

COMPANY INTRODUCTION

SHENai Private Software Solutions is a software and technology services company located in Perundurai, Erode, Tamil Nadu. The company focuses on providing innovative software solutions by combining technology, creativity, and customer requirements. It aims to help businesses and individuals improve their operations through modern digital technologies.

The company offers services in Artificial Intelligence, Data Science, web development, mobile application development, UI/UX design, e-commerce development, office automation, Internet of Things (IoT), and robotics. It focuses on developing customized and scalable solutions based on client needs.

SHENai Private Software Solutions emphasizes innovation, quality, teamwork, and customer satisfaction. The company also provides internship opportunities that help students gain exposure to industry practices and develop technical knowledge in different technology domains.

VISION AND MISSION

VISION

The vision of SHENai Private Software Solutions is to be at the forefront of AI-based digital transformation by developing innovative and reliable software solutions. The company aims to encourage technological advancement and provide solutions that address the changing needs of businesses and individuals.

MISSION

The mission of SHENai Private Software Solutions is to empower businesses and individuals through next-generation software solutions. The company focuses on delivering customized technology services, maintaining quality, encouraging innovation, and developing solutions that support business growth and operational efficiency.

PRODUCTS/SERVICES

Task Pilot

TaskPilot is a mobile application publicly described by SHENai Private Software Solutions for helping small businesses manage their daily activities. It supports task management, daily checklists, reminders, and work progress tracking. The application is designed to simplify routine business operations and improve task organization. It supports Tamil and English, making it accessible to a wider range of users.

SERVICES

1. Artificial Intelligence and Data Science:

Developing AI-based solutions, data analysis, predictive analytics, and intelligent automation.

2. Web Development:

Designing and developing responsive websites and web applications based on customer requirements.

3. Mobile Application Development:

Creating mobile applications to provide convenient access to digital services.

4. UI/UX Design:

Designing user-friendly interfaces and improving the usability and overall experience of digital applications.

5. Internet of Things (IoT):

Developing technology-based solutions that connect devices and sensors for monitoring and automation.

6. Robotics:

Exploring robotic systems and automation technologies .

INDUSTRY DOMAIN

SHENai Private Software Solutions operates in the Information Technology (IT) and software development industry, with a focus on Artificial Intelligence and emerging technologies. The IT industry plays an important role in developing digital solutions that help organizations manage information, automate processes, and improve productivity.

Artificial Intelligence is one of the company's key technology areas. AI enables computer systems to perform tasks such as recognizing patterns, analysing data, generating information, and supporting decision-making. Data Science complements AI by helping extract useful insights from data.

In addition to AI and Data Science, the company provides services related to web and mobile application development, UI/UX design, e-commerce, office automation, IoT, and robotics. These technologies can be applied across different sectors to address business and operational requirements.

The company's industry domain provides opportunities for students to understand the practical applications of modern technologies and gain exposure to software development practices.

DEPARTMENT AND TEAM ASSIGNED

The AI domain focuses on understanding how intelligent systems can be designed to analyze information, identify patterns, automate tasks, and solve problems. Learning in this domain can involve exploring AI concepts, understanding development tools, analyzing requirements, and studying the implementation of technology-based solutions, depending on the activities assigned.

The team environment in a software organization supports knowledge sharing, collaboration, and guidance during technical activities. Team members may contribute through planning, design, development, and coordination according to their responsibilities.

SHENai's published team information identifies Senthamarai M as Founder and CEO, Sandeep S D as Director of Planning and Development, and Rithik P as Director of Design. However, the specific mentor or team directly assigned to an intern depends on the internship arrangement.

My internship experience in the AI domain helped provide an opportunity to connect academic knowledge with industry-oriented learning and develop an understanding of professional technology practices.

KEY FEATURES OF AN ORGANISATION

SHENai Private Software Solutions focuses on providing technology-based solutions and software services. The key features of the organization are as follows:

1. Focus on Artificial Intelligence:

The organization provides AI-related solutions and explores the use of intelligent technologies to solve practical problems.

2. Diverse Technology Services:

It offers services in Artificial Intelligence, Data Science, web development, mobile application development, UI/UX design, and e-commerce development.

3. Innovation and Technology:

The organization emphasizes the use of modern technologies to develop solutions based on customer requirements.

4. Customized Solutions:

Its software services are designed to address different business needs and support digital transformation.

5. Automation:

The organization includes office automation, IoT, and robotics among its technology service areas, supporting the development of automated solutions.

6. User-Oriented Applications:

Its application development and UI/UX design services focus on usability and providing convenient digital experiences.

7. Practical Learning Opportunities:

Internship opportunities can help students gain exposure to AI concepts, software development practices, and industry-oriented learning.

INTERNSHIP ACTIVITIES

INTERNSHIP PLAN

The internship plan was designed to provide practical knowledge of Artificial Intelligence and its applications in the software industry. The initial stage focused on understanding the fundamentals of AI and its importance in solving real-world problems. This was followed by learning Python programming and exploring the tools commonly used in AI development.

The next stage focused on understanding data processing, basic Machine Learning concepts, and the development of AI-based solutions. The final stage involved testing, reviewing the results, and documenting the learning

experience. This planned approach helps connect theoretical knowledge with practical applications and improves technical understanding.

TRAINING RECEIVED

1. Introduction to Artificial Intelligence

Training included an introduction to AI concepts, applications, and the role of intelligent systems in solving real-world problems.

2. Python Programming

Python programming concepts relevant to AI development were explored, including variables, data types, conditional statements, loops, functions, and libraries.

3. Data Processing and Analysis

The training covered the importance of preparing data before using it in AI applications. This included understanding data cleaning, handling missing values, organizing datasets, and basic data analysis.

4. Machine Learning Fundamentals

Basic concepts of Machine Learning were studied, including supervised learning, unsupervised learning, training data, testing data, and model evaluation.

5. AI Development Tools

Exposure to programming environments and libraries used for AI development helped develop an understanding of how AI experiments and applications can be implemented.

6. Testing and Documentation

The training also covered the importance of testing solutions, identifying errors, recording observations, and documenting the development process.

TECHNOLOGIES AND TOOLS USED

During the internship in the Artificial Intelligence domain, various programming languages, frameworks, libraries, and AI technologies were explored for developing AI-based applications. These technologies support application development, code analysis, user interface creation, and integration with Large Language Models (LLMs).

Python: Python is a high-level programming language widely used in Artificial Intelligence and software development. It provides a simple syntax and extensive libraries for developing applications, processing data, and implementing AI functionalities.

FastAPI: FastAPI is a Python web framework used to develop APIs. It helps connect the application interface with backend functionality and supports efficient communication between different components of a software system.

Streamlit: Streamlit is a Python framework used to create interactive web applications. It helps developers build user-friendly interfaces for AI applications without requiring extensive frontend development.

AST (Abstract Syntax Tree): AST is a Python module used to analyze the structure of Python source code. It represents code as a structured tree and can help inspect functions, classes, and other programming elements.

Groq API: The Groq API provides access to supported Large Language Models through an API. It can be integrated into applications to generate text, process prompts, and produce responses based on user input.

LLM (Large Language Model): An LLM is an AI model trained on large amounts of text to understand and generate human-like language. In AI applications, LLMs can be used for tasks such as text generation, question answering, summarization, and code documentation.

Uvicorn: Uvicorn is an ASGI server used to run Python web applications, including FastAPI applications. It receives application requests and helps serve responses to clients.

Git: Git is a distributed version control system used to track changes in source code. It helps developers maintain different versions of a project, review modifications, and manage code efficiently during software development.

GitHub: GitHub is a web-based platform used to store, manage, and share source code repositories. It supports collaboration among developers and helps maintain project files, documentation, and version history using Git.

Antigravity: Antigravity is an AI-powered development platform that assists developers in creating and modifying software applications using AI-based coding assistance. It can help with code generation, debugging, project development, and improving development productivity

Python Libraries: Python provides a wide range of libraries that simplify application development and reduce the need to write code from scratch. Libraries are reusable collections of code that provide specific functionalities

Integration of Technologies

These technologies can work together to develop an AI-powered application. Python provides the core programming logic, while FastAPI handles backend API operations. Streamlit provides the user interface, and the Groq API enables integration with supported LLMs. AST can be used when the application

requires Python source-code analysis, while Uvicorn runs the FastAPI backend server.

PHASE-WISE ACTIVITIES

Phase 1: Introduction to AI and Generative AI

The initial phase focused on understanding the fundamentals of Artificial Intelligence and Generative AI. Concepts such as Large Language Models (LLMs), prompt engineering, text generation, and AI-based automation were explored. This phase provided a foundation for understanding how AI technologies can be integrated into software applications.

Phase 2: Python Programming and Code Analysis

The second phase focused on Python programming and its applications in AI development. Basic programming concepts and Python libraries were explored. The AST module was studied to understand how Python source code can be analyzed and represented as a structured tree. This concept is useful for applications such as automated Python docstring generation.

Phase 3: Backend Development Using FastAPI

The third phase focused on developing backend functionality using FastAPI. The framework was explored to understand how APIs are created and how requests and responses are handled. Uvicorn was

used to run the FastAPI application and make the backend available for communication with other application components.

Phase 4: Frontend Development Using Streamlit

This phase focused on creating interactive user interfaces using Streamlit. The framework was used to understand how users can enter information, submit requests, and view results through a web application. The interface can be connected to backend functionality to provide a simple and accessible user experience.

Phase 5: Generative AI Integration Using Groq API

The fifth phase focused on integrating supported Large Language Models through the Groq API. The process involved understanding API requests, preparing prompts, sending user inputs to the model, and displaying generated responses. This integration supports AI functionalities such as text generation, code documentation, and question answering, depending on the application requirements.

Phase 6: Application Development Using Antigravity

Antigravity was explored as an AI-assisted development tool to support application development. It can assist with generating code, understanding project requirements, modifying application components, and identifying potential errors. Its use supports the development workflow while requiring the generated code to be reviewed and tested.

Phase 7: Version Control Using Git and GitHub

The final development phase focused on version control and project management. Git was used to understand how code changes can be tracked and managed, while GitHub provides a platform for storing and sharing project repositories. These tools support code organization, version history, and collaboration during software development.

Phase 8: Testing and Final Review

The final phase focused on reviewing the application components and understanding their integration. Python code, FastAPI endpoints, Streamlit interfaces, and Groq API responses were checked according to the application requirements. Errors were identified and corrected where necessary, and the development activities were documented for future reference.

TEAM COLLABORATION

Team collaboration is an important part of the software development process. During the internship in the Artificial Intelligence domain at SHENai Private Software Solutions, Erode, collaboration helps in understanding technical concepts, discussing development approaches, and learning about AI-based application development.

Communication with mentors and team members provides opportunities to clarify doubts, share ideas, and receive feedback on technical tasks. Tools such as Git and GitHub support code management and collaboration by helping track

changes and maintain project files. AI-assisted development tools such as Antigravity can also support coding activities and debugging.

Team collaboration encourages knowledge sharing, problem-solving, and effective task management. It also helps develop professional skills such as communication, teamwork, time management, and adaptability. These skills are valuable for working effectively in a professional software development environment.

MAJOR PROJECT

PROJECT TITLE

AUTOMATED PYTHON DOCSTRING GENERATOR

The project focuses on developing an AI-powered application that automatically generates professional docstrings for Python source code. It uses Abstract Syntax Tree (AST) parsing and Large Language Models (LLMs) to understand code structure and generate meaningful documentation. The application also provides code explanation, and code optimization features to improve code readability and developer productivity.

PROBLEM STATEMENT

Documentation is an essential part of software development because it helps developers understand the purpose, functionality, and usage of source code. However, writing docstrings manually for every function, method, and class can be time-consuming and repetitive. Developers may skip documentation because of limited time or project deadlines.

Insufficient documentation makes code difficult to understand, maintain, debug, and share among team members. Manual documentation may also be inconsistent in format and quality.

Therefore, an automated system is required to analyse Python source code, understand its structure, and generate appropriate docstrings. The proposed system aims to reduce documentation effort and improve code readability through AI-based automation.

OBJECTIVE OF THE PROJECT

The primary objective of this project is to automate Python code documentation using Artificial Intelligence.

The specific objectives are to:

1. Automatically generate docstrings for Python functions, methods, and classes.
2. Use AST to identify code structures and extract relevant information.
3. Integrate an LLM through the Groq API to generate meaningful documentation.
4. Support multiple documentation formats, including Google, NumPy, and Sphinx styles.
5. Provide simple explanations of Python code to support learning and debugging.
6. Offer code optimization suggestions and comparisons between original and optimized code.
7. Provide an interactive interface for uploading Python files and viewing generated results.
8. Reduce manual effort and improve the consistency and readability of code documentation.

REQUIREMENTS

The project requires suitable hardware and software to develop, execute, and test the application.

Hardware Requirements

A computer or laptop capable of running Python development tools and a web browser is required. A stable internet connection is needed for communication with the Groq API.

Software Requirements

The main software requirements are Python, Streamlit, FastAPI, Uvicorn, and the Groq API. A code editor such as Visual Studio Code can be used for development. Git and GitHub can support version control and source-code management.

Functional Requirements

The application should allow users to upload Python source files, analyze their code structure, generate docstrings, and download the updated source code. It should also support different documentation styles and provide code explanations and optimization suggestions.

Non-Functional Requirements

The application should be user-friendly, responsive, and capable of processing valid Python files reliably. It should validate inputs, handle API errors, maintain the original code structure as far as possible, and ensure that generated documentation does not introduce unsupported descriptions of the code.

EXISTING SYSTEM

In the traditional approach, developers manually write docstrings for functions, methods, and classes. They must understand the code, identify parameters and return values, and describe the functionality using an appropriate documentation format.

This process requires time and effort, especially in projects containing many functions and complex code structures. Documentation may also become outdated when the source code changes.

The manual approach can result in inconsistent documentation and reduced productivity. These limitations create a need for an automated system that can assist developers in generating and maintaining code documentation.

PROPOSED SOLUTION

The proposed system is an AI-powered Python docstring generator that combines AST-based code analysis with an LLM accessed through the Groq API.

The user uploads a Python file through the Streamlit interface. The backend uses Python and AST to identify functions, methods, classes, parameters, and other relevant code structures. The extracted information and source code are then processed using an AI model to generate suitable docstrings.

The application validates the generated response and inserts the documentation into the appropriate positions in the source code. The updated code is then made available to the user.

The system also includes code explanation and optimization features. Code explanation provides a simplified description of the program's logic, while the

optimization feature suggests improvements and displays an original-versus-optimized comparison.

This approach reduces repetitive documentation work and provides additional assistance for understanding and improving Python code.

METHODOLOGY

The project follows a systematic development methodology consisting of input processing, code analysis, AI-based generation, validation, and output generation.

Step 1: Input and Validation

The user uploads a Python source file through the Streamlit interface. The application checks the input before sending it for processing.

Step 2: AST-Based Code Analysis

The Python AST module parses the source code into a structured representation. This allows the application to identify functions, classes, methods, parameters, and other relevant elements.

Step 3: AI-Based Documentation Generation

The extracted code information is sent to a supported LLM through the Groq API. Carefully designed prompts guide the model to generate meaningful docstrings in the selected documentation style.

Step 4: Response Processing and Validation

The AI-generated response is cleaned and validated. The project presentation identifies mechanisms for cleaning invalid JSON, checking AI output against AST information, normalizing function names, and retrying failed responses.

Step 5: Docstring Insertion

The generated documentation is inserted into the appropriate locations in the source code while maintaining indentation and code structure.

Step 6: Output Generation

The application returns the updated Python file to the user. The interface also supports viewing code explanations and comparing original code with suggested optimized code.

SYSTEM ARCHITECTURE

The system follows a frontend-backend architecture with an AI integration component. The frontend accepts user input and displays the generated output, while the backend handles source-code processing and communication with the language model.

The architecture consists of the following components:

Frontend Layer – Streamlit: Provides the user interface for uploading Python files, selecting documentation styles, viewing results, and accessing additional features.

Backend Layer – FastAPI: Handles requests from the frontend and coordinates code analysis, AI processing, validation, and output generation.

Code Analysis Layer – Python AST: Parses the source code and extracts information about functions, classes, methods, and parameters.

AI Processing Layer – Groq API and LLM: Processes prompts and generates docstrings, code explanations, and optimization suggestions.

Execution Layer – Uvicorn: Runs the FastAPI application as an ASGI server.

Output Layer: Returns the documented source code and other generated results to the user.

The overall workflow is:

Python File Upload → Input Validation → AST Parsing → LLM Processing through Groq API → Response Validation → Docstring Insertion → Updated Python File.

MODULES

File Upload and Validation Module

This module allows users to upload Python source files through the interface. It validates the input and prepares the file for further processing. This helps ensure that the system receives an appropriate file before beginning code analysis.

AST-Based Code Parsing Module

This module uses Python's AST functionality to analyse the source code. It identifies functions, classes, methods, and parameters, providing structured information that supports documentation generation.

AI-Based Docstring Generation Module

This module communicates with an LLM through the Groq API. The model uses the source code and relevant structural information to generate documentation describing the purpose of functions and their parameters and return values.

The system supports Google, NumPy, and Sphinx documentation styles, allowing generated docstrings to follow different formatting conventions.

Docstring Insertion Module

This module inserts the generated docstrings into the appropriate positions in the Python source code. It considers indentation and code structure to help preserve the validity of the updated file.

Code Explanation Module

The code explanation module converts programming logic into a simpler description. It provides step-by-step explanations and examples where appropriate, helping users understand code and supporting learning and debugging.

Code Optimization Module

This module generates suggestions for improving source code. It presents a comparison between the original code and the suggested optimized version, together with a description of the proposed improvements.

Output and Integration Module

This module displays the generated documentation and provides the updated Python file. The project presentation also identifies Visual Studio Code integration, allowing the tool to send code to the FastAPI backend and receive documented code within the editor.

IMPLEMENTATION

The application was designed using Python as the core programming language. Streamlit provides the user interface, while FastAPI manages backend operations. Uvicorn runs the backend server, and the Groq API connects the application to a supported Large Language Model.

During implementation, the uploaded Python source code is parsed using AST to identify important code structures. This information supports prompt

construction and helps the AI model generate documentation that corresponds to the actual source code.

The generated response is processed before being inserted into the source code. According to the project presentation, response-cleaning mechanisms, validation against AST information, function-name normalization, and retry handling were implemented to address common AI response errors.

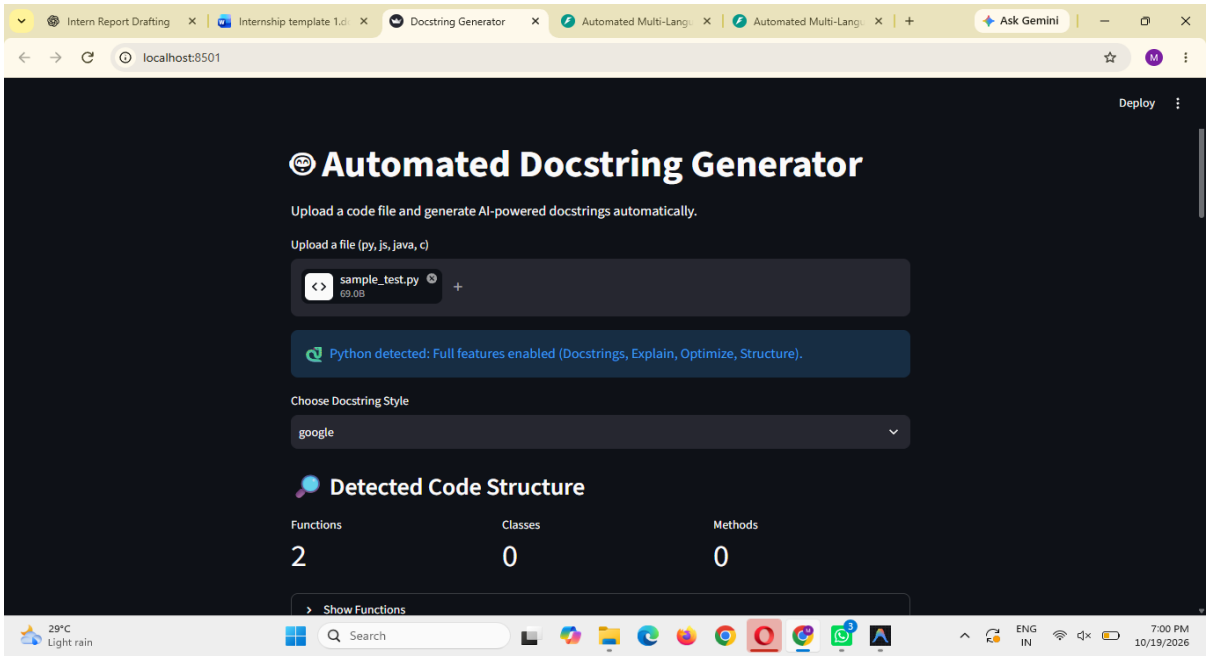
The application was also designed to support multiple docstring styles, simplified code explanations, and code optimization suggestions. The Visual Studio Code integration described in the presentation provides direct access to documentation generation from the editor.

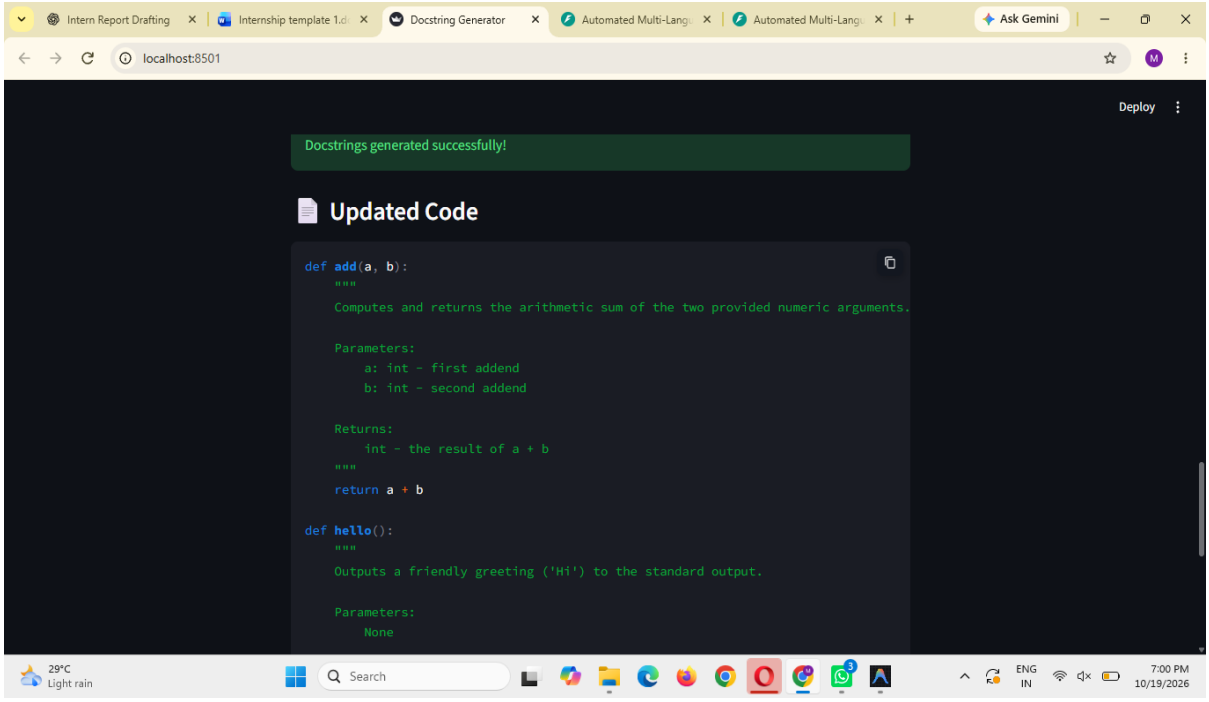
These components work together to provide an automated documentation workflow through an interactive interface.

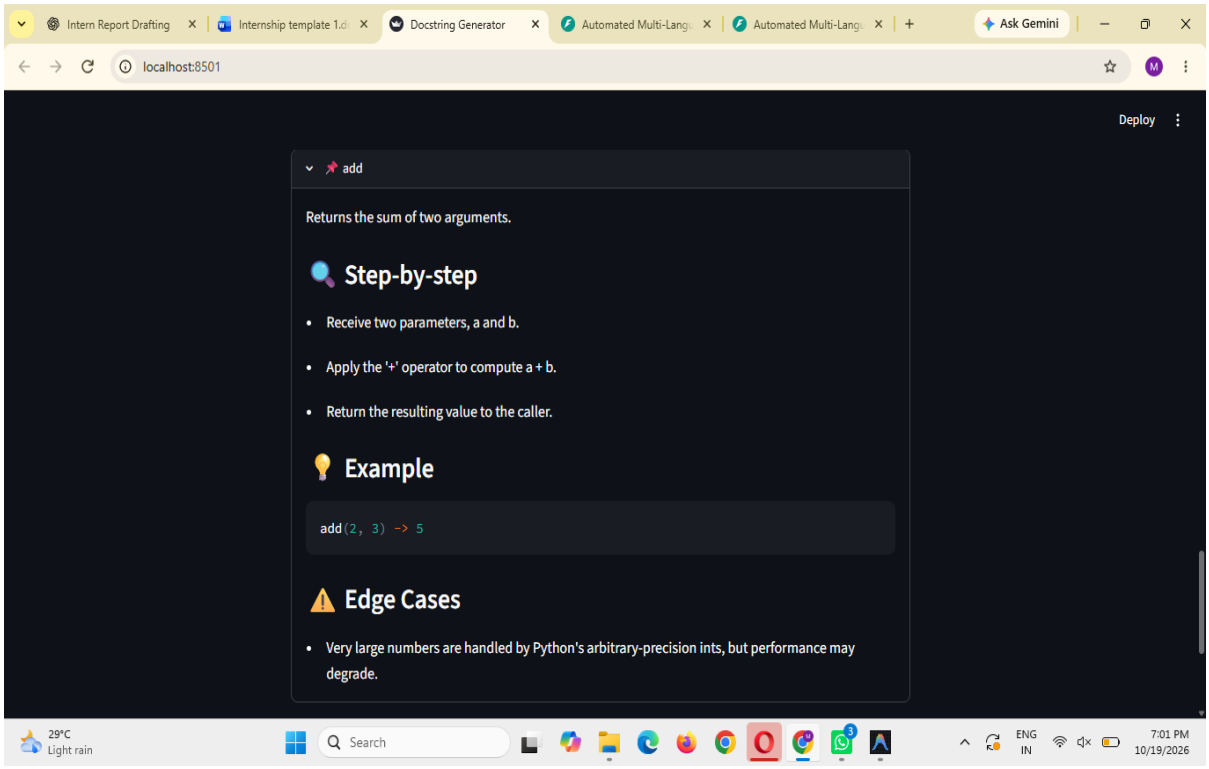
RESULTS AND OUTPUTS

The project presentation reports that the automated Python docstring generator successfully generates documentation for Python functions and methods and supports Google, NumPy, and Sphinx styles. It also includes code explanation, code optimization, and Visual Studio Code integration.

The main output is an updated Python source file containing generated docstrings. The system also provides explanations of code logic, examples where appropriate, and comparisons between original and optimized code.


Intern Report Drafting
Internship template 1.d
Docstring Generator
Automated Multi-Lang
Automated Multi-Lang
Ask Gemini
localhost:8501
Deploy
Automated Docstring Generator
Upload a code file and generate AI-powered docstrings automatically.
Upload a file (py, js, java, c)
sample_test.py
69.0B
Python detected: Full features enabled (Docstrings, Explain, Optimize, Structure).
Choose Docstring Style
google
Detected Code Structure
Functions
2
Classes
0
Methods
0
Show Functions
29°C
Light rain
Search
ENG
IN
7:00 PM
10/19/2026


Intern Report Drafting
Internship template 1.d
Docstring Generator
Automated Multi-Lang
Automated Multi-Lang
Ask Gemini
localhost:8501
Deploy
Docstrings generated successfully!
Updated Code
def add(a, b):
"""
Computes and returns the arithmetic sum of the two provided numeric arguments.
Parameters:
a: int - first addend
b: int - second addend
Returns:
int - the result of a + b
"""
return a + b
def hello():
"""
Outputs a friendly greeting ('Hi') to the standard output.
Parameters:
None
29°C
Light rain
Search
ENG
IN
7:00 PM
10/19/2026



SKILLS AND LEARNING OUTCOMES

TECHNICAL SKILLS

- Python Programming: Learned to write Python programs, develop functions, handle errors, and process code.
- AST (Abstract Syntax Tree): Learned to analyze Python code structures, including functions, classes, methods, and parameters.
- FastAPI: Gained knowledge of developing backend APIs to process requests and return responses.
- Streamlit: Learned to develop an interactive user interface for uploading Python files and displaying generated documentation.
- Groq API and LLM: Gained practical exposure to integrating a Large Language Model for AI-powered docstring generation.
- Prompt Engineering: Learned to design prompts to generate meaningful and structured documentation.

- Code Validation: Practiced validating AI responses and handling incorrect or incomplete outputs.

SOFTWARE/TOOLS/PLATFORMS

- Visual Studio Code: Used as a development environment for writing and editing Python code.
- Python: Used as the primary programming language.
- FastAPI: Used to develop the backend service.
- Streamlit: Used to build the interactive frontend.
- Uvicorn: Used to run the FastAPI application server.
- Git and GitHub: Used for version control and managing project source code.
- Antigravity: Used as an AI-assisted development tool.
- Groq API: Used to connect the application with an LLM for generating docstrings.
- Python Libraries: Used relevant libraries to support code processing, API communication, and application development.

AI/CLOUD/DATASKILLS

- Generative AI: Learned how AI generates text-based outputs, such as Python docstrings and code explanations.
- Large Language Models (LLMs): Gained knowledge of using language models to interpret code and generate documentation.
- Prompt Engineering: Learned to structure prompts to improve the relevance and format of AI-generated responses.
- API Integration: Practiced connecting an application to an external AI service using the Groq API.

- Data Processing and Validation: Learned to extract code information, process AI responses, and validate generated outputs.

COMMUNICATION AND TEAMWORK

- Improved the ability to explain technical concepts in simple language.
- Developed skills in presenting project objectives, features, workflow, and results.
- Learned the importance of sharing ideas and discussing technical challenges.
- Understood the value of clear communication during software development.
- Developed the ability to document project activities and explain implementation details.
- Recognized the importance of coordination and feedback in completing technical tasks.

PROBLEM SOLVING SKILLS

- Error Handling: Learned to identify and handle invalid or incomplete AI-generated responses.
- Debugging: Practiced identifying errors and correcting problems during implementation.
- Response Validation: Applied validation techniques to compare AI-generated documentation with the actual Python code structure.
- Function Matching: Learned to match generated documentation with the appropriate functions and methods.
- Accuracy Improvement: Understood the importance of verifying parameter details, return information, and code explanations.

- Logical Thinking: Developed a systematic approach to analyzing problems, testing solutions, and improving application reliability.

PROFESSIONAL LEARNING

- Improved the ability to learn new technologies independently.
- Gained experience in reading technical documentation and understanding API usage.
- Learned to break a large project into smaller modules and development stages.
- Developed the habit of testing and validating application outputs.
- Understood the importance of code quality, documentation, and maintainability.
- Gained exposure to AI-assisted development tools and modern software development practices.
- Improved the ability to adapt to new tools, technologies, and technical requirements.

CHALLENGES AND SOLUTIONS

TECHNICAL CHALLENGES

- Invalid AI Responses:
  The Groq API sometimes returned responses in an unexpected JSON format.
- AI Hallucinations:
  The generated documentation sometimes contained incorrect or unsupported explanations of the code.
- Function Matching:

Matching AI-generated docstrings with the correct functions and methods was challenging.

- Parameter and Return Accuracy:
  Ensuring that parameter names, return values, and descriptions matched the actual code required careful validation.
- Nested Functions and Class Methods:
  Processing nested functions and methods within classes required accurate code structure analysis.
- Docstring Insertion:
  Inserting docstrings at the correct positions while maintaining proper indentation was an important challenge.

ORGANISATIONAL AND TEAM CHALLENGES

- Task Planning: Organizing project activities into manageable development stages.
- Technical Understanding: Understanding the project requirements and selecting suitable technologies.
- Coordination: Communicating technical ideas and discussing possible solutions with team members or the mentor, where applicable.
- Time Management: Completing development, testing, and documentation within the available time.
- Integration: Ensuring that the frontend, backend, and AI components worked together correctly.

SOLUTION ADOPTED

- JSON Cleaning:
  Applied response-cleaning logic to handle incorrectly formatted AI responses.
- Strict Validation:
  Compared the AI-generated output with information extracted from the Python AST.
- Smart Cleaning Logic:
  Processed the generated content to remove unwanted formatting and improve output consistency.
- Name Normalization:
  Used normalization techniques to improve matching between functions and generated documentation.
- Retry Mechanism:
  Implemented retries to handle certain unsuccessful AI responses.
- AST-Based Analysis:
  Used AST parsing to identify functions, classes, methods, and parameters before inserting docstrings.

KEY LESSONS LEARNED

The internship provided valuable knowledge about AI application development and software engineering practices through the Automated Python Docstring Generator project. It helped in understanding how Python programming can be used to develop practical applications and automate repetitive tasks. The project also highlighted the importance of writing clean, readable, and maintainable code.

Another important lesson learned was the use of Python's Abstract Syntax Tree (AST) to analyze code structures, including functions, classes, methods, and parameters. The integration of the Groq API and Large Language Models

(LLMs) provided an understanding of how AI can generate meaningful documentation and explain code. Prompt engineering was also an important concept learned during the development process.

PRACTICAL EXPERIENCE GAINED

The internship provided hands-on experience in developing the Automated Python Docstring Generator using Python, Streamlit, FastAPI, Uvicorn, AST, and the Groq API. Python was used for the core programming logic, while Streamlit supported the interactive user interface. FastAPI and Uvicorn were used to develop and run the backend application, providing practical knowledge of how different components work together in a software system.

Practical knowledge was gained in analyzing Python source code using AST and identifying functions, classes, methods, and parameters. The project also provided experience in integrating a Large Language Model through the Groq API to generate meaningful docstrings. Different documentation formats, including Google, NumPy, and Sphinx, were explored to make the generated documentation useful for different coding requirements.

PROFESSIONAL EXPERIENCE

WORKPLACE PRACTICES

The internship provided an understanding of professional workplace practices followed in software development. Working on the Automated Python Docstring Generator helped develop a systematic approach to task planning, coding, testing, and project documentation. It also highlighted the importance of discipline, time management, responsibility, and continuous learning in completing project activities effectively.

Key Workplace Practices Followed:

- Maintained an organized approach to project development.
- Divided the project into smaller modules for easier implementation.
- Used appropriate development tools such as Python, VS Code, and Antigravity.
- Tested application components and corrected errors during development.
- Maintained clear project documentation.
- Focused on completing tasks within the planned timeframe.
- Practiced continuous learning to understand new technologies and tools.

Industry standards

The project provided practical exposure to software development practices that support code quality, consistency, reliability, and maintainability. The use of Python AST for code analysis and Google, NumPy, and Sphinx docstring formats demonstrated the importance of structured code documentation. The integration of Streamlit, FastAPI, and the Groq API also provided an understanding of how different technologies can work together in an AI-powered application.

Key Industry Practices Learned:

- Followed structured and readable Python coding practices.
- Used standard docstring formats for consistent documentation.
- Applied input validation and error-handling techniques.
- Tested generated outputs to identify possible errors.

PROFESSIONAL ETHICS

Professional ethics are essential for developing trustworthy and responsible software applications. During the internship, the importance of accuracy, honesty, accountability, and responsible AI usage was understood. Since the project generates documentation using an LLM, reviewing the output before accepting it is important to avoid misleading explanations and incorrect code descriptions.

Key Professional Ethics Learned:

- Maintained honesty while documenting project work and reporting results.
- Recognized the importance of verifying AI-generated content.
- Respected intellectual property and software licensing requirements.
- Used development tools and external APIs responsibly.

SAFETY,SECURITY AND DATA PRACTICES

The internship highlighted the importance of secure development and responsible data handling in AI applications. The Automated Python Docstring Generator processes uploaded Python files and communicates with an external AI service through the Groq API. Therefore, validating uploaded files, protecting API credentials, and reviewing generated outputs are important practices for improving application security and reliability.

Key Safety, Security, and Data Practices:

- Validate uploaded Python files before processing them.
- Handle invalid inputs and unexpected responses appropriately.
- Protect API keys and avoid exposing credentials in public repositories.
- Avoid sharing confidential source code with external AI services without authorization.

- Handle uploaded files and generated outputs responsibly.
- Review AI-generated documentation for correctness.

CONCLUSION AND FUTURE SCOPE

CONCLUSION

The internship at SHENai Private Software Solutions provided valuable knowledge and practical exposure to Artificial Intelligence and software development. Through the development of the Automated Python Docstring Generator, the concepts learned in Python programming, AI integration, backend development, and code analysis were applied to a practical project. The internship helped understand how AI technologies can be used to automate repetitive programming tasks and improve software documentation.

ACHIEVEMENTS OF THE OBJECTIVES

The main objectives of the internship were addressed through practical project development and the application of AI technologies. The project helped demonstrate how automated documentation can reduce repetitive manual work and support better code readability and maintainability.

- Python Programming: Applied Python concepts to develop the core application.
- AI Integration: Integrated the Groq API and an LLM for AI-powered docstring generation.
- Code Analysis: Used AST to identify Python functions, classes, methods, and parameters.
- Application Development: Developed an interactive interface using Stream lit and backend functionality using Fast API.

- Documentation Generation: Implemented support for Google, NumPy, and Sphinx docstring formats.
- Error Handling: Applied validation and response-cleaning techniques to improve output reliability.
- Practical Learning: Gained experience in debugging, testing, API integration, and software development tools.

CAREER RELEVALANCE

The internship experience is relevant to career opportunities in Artificial Intelligence, Python development, and software engineering. Developing an AI-powered application provided practical exposure to technologies used in building intelligent software systems. The knowledge gained can support preparation for entry-level technical roles and future projects.

The following skills are relevant to professional career development:

- AI Engineer: Understanding LLM integration, prompt engineering, and AI-powered automation.
- Python Developer: Applying Python programming, AST analysis, debugging, and code validation.
- Software Developer: Developing applications using frontend and backend technologies.
- Generative AI Developer: Integrating language models into applications through APIs.
- Backend Developer: Building API services using FastAPI and running applications with Uvicorn.

The internship also provided a foundation for building a professional portfolio, presenting project work during interviews, and demonstrating practical knowledge of AI application development.

FUTURE SCOPE

The Automated Python Docstring Generator can be enhanced with additional features to improve its functionality, scalability, and usefulness for software developers. The following improvements could be considered in future development.

- Multi-Language Support: Extend documentation generation to programming languages such as Java, JavaScript, and C++.
- Multi-File Upload: Allow users to upload and process multiple Python files simultaneously.
- Automatic Documentation Updates: Update docstrings whenever the associated code changes.
- Advanced Code Optimization: Provide more detailed suggestions for improving code readability, performance, and maintainability.
- Improved AI Accuracy: Enhance prompt engineering, validation, and testing to reduce incorrect AI-generated documentation.
- Additional Documentation Formats: Support more documentation styles and customization options.

REFERENCES

1. Python Software Foundation, Python Documentation. Available at: https://docs.python.org/3/

2. Python Software Foundation, AST— Abstract Syntax Trees. Available at: https://docs.python.org/3/library/ast.html

3. FastAPI, FastAPI Documentation:https://fastapi.tiangolo.com/

INTERNSHIP CERTIFICATE



SHENai
Private Software Solutions

Ministry of MSME, Govt. of India
UDYAM-TN-07-0077872

CERTIFICATE ID : SHEN-2026-INT-0001

CERTIFICATE

OF INTERNSHIP

This is to certify that

MADHUSHREE M

student of **SSM INSTITUTE OF ENGINEERING AND TECHNOLOGY DINDIGUL** pursuing a B.Tech in Artificial Intelligence and Data Science, has successfully completed internship in Artificial Intelligence at SHENai Private Software Solutions from [05/07/2026] to [05/09/2026].

During this time, HE/SHE displayed excellent design skills, creativity, and teamwork, contributing significantly to our projects.

M. Senthamarai

SENTHAMARAI M
FOUNDER & CEO